

A Technical Deep Dive: OpenFaaS Classic Watchdog vs. of-watchdog in Serverless Architectures

1. Introduction to OpenFaaS Watchdogs

OpenFaaS stands as a prominent open-source serverless platform, meticulously engineered to streamline the deployment of event-driven functions and microservices atop Kubernetes. This platform empowers developers by enabling them to encapsulate their application logic within Docker images, subsequently exposing these as highly scalable endpoints equipped with integrated auto-scaling and comprehensive metrics capabilities.¹ The architectural elegance of OpenFaaS is significantly underpinned by its "watchdog" component, a critical element residing within each function container.

Functioning as an "init process" and an embedded HTTP server, typically implemented in Golang, the watchdog serves as the indispensable intermediary between the OpenFaaS Gateway and the function's core business logic.² Its fundamental responsibilities encompass receiving incoming HTTP requests, meticulously marshaling them into a format consumable by the function, and subsequently invoking the user's defined application.⁴ This report undertakes a comprehensive technical examination of the two principal watchdog implementations within the OpenFaaS ecosystem: the foundational Classic Watchdog and its more evolved successor, the of-watchdog. The analysis will meticulously delineate their core architectural distinctions, with a particular emphasis on their respective approaches to handling concurrent requests within a pod, their underlying concurrency models, and the specific mechanisms, such as `max_inflight`, that govern their behavior. The objective is to furnish technical professionals, including senior software engineers, cloud architects, and DevOps specialists, with the profound understanding necessary to make informed architectural decisions, optimize function performance, and effectively diagnose and resolve issues in complex serverless deployments on OpenFaaS.

2. The Classic Watchdog: Design and Concurrency Model

Core Design Principles: STDIO Communication and Process-Per-Request Model

The Classic Watchdog was engineered with paramount considerations for portability and simplicity, enabling the encapsulation of virtually any binary or script as a serverless function. Its foundational communication paradigm hinges on standard input/output (STDIO), where the

HTTP request body is transmitted via stdin to the function, and the function's response is subsequently captured from stdout.⁴ This design effectively transforms a conventional command-line interface (CLI) or a straightforward script into a functional HTTP REST API endpoint.⁸

A defining characteristic of the Classic Watchdog's operational model is its strict "process-per-request" approach. For every incoming HTTP request, the watchdog initiates and manages a new, isolated process dedicated solely to the execution of the user's function.³ This methodology bears a strong conceptual resemblance to the Common Gateway Interface (CGI), a historical standard in web server technology.⁷ The watchdog itself is a compact web server, implemented in Golang, which listens on port 8080 within the container. Its role involves handling the intricacies of the HTTP protocol, marshaling the request components (such as headers and the body) into the stdin of the newly forked function process, and subsequently capturing the stdout of that process to construct the HTTP response transmitted back to the client.⁴

Concurrency Handling: How it Manages Multiple Simultaneous Requests, Including the `max_inflight` Parameter

Although the Classic Watchdog's embedded Golang HTTP server is inherently capable of accepting and managing concurrent incoming HTTP connections, its approach to executing the user's function remains strictly bound by the "process-per-request" paradigm. This implies that for each concurrent request received, a distinct function process is spawned and subsequently managed by the watchdog.² Each invocation benefits from a high degree of process-level isolation, ensuring that one function's execution does not directly interfere with another's within the same container.¹¹

To prevent a single function pod from becoming overwhelmed by an excessive number of concurrent requests, the Classic Watchdog incorporates the `max_inflight` environment variable. This parameter enables system administrators to impose a hard limit on the maximum number of concurrent requests that the watchdog will actively process within that specific pod.² Should this configured limit be reached, any subsequent incoming requests will be gracefully rejected by the watchdog with an HTTP 429 Too Many Requests status code. This behavior acts as a crucial backpressure mechanism, signaling to the upstream caller—such as the OpenFaaS Gateway or a client application equipped with retry logic—that the function is currently operating at its maximum capacity.¹³ Furthermore, the `max_inflight` value can be leveraged by the OpenFaaS autoscaler to inform capacity-based scaling decisions, allowing the platform to dynamically adjust the number of function replicas based on observed load.¹³

Ideal Use Cases and Inherent Limitations

The Classic Watchdog finds its ideal application in scenarios involving functions developed in programming languages or environments that do not natively expose an HTTP server. This includes, but is not limited to, Bash scripts, PowerShell scripts, or even legacy COBOL

programs.⁸ It demonstrates particular efficacy in wrapping existing command-line tools or simple, stateless scripts, transforming them into accessible HTTP endpoints.⁸ Its inherent simplicity also renders it a suitable choice for initial development, educational tutorials, or batch processing jobs where peak performance is not the primary concern.⁶

Despite its utility, the Classic Watchdog carries several inherent limitations. The most notable is the overhead associated with spawning a new process for every single request. This leads to discernible "cold start" latencies, a phenomenon particularly pronounced for languages characterized by slower startup times, such as Java or Node.js.⁷ Empirical observations suggest that cold starts can account for a substantial portion—up to 80%—of the total invocation latency in serverless environments.¹⁷ Furthermore, the continuous creation and termination of processes for each request can result in higher CPU and memory consumption per invocation when compared to alternative models that facilitate the reuse of existing processes.¹⁸ Performance benchmarks consistently highlight that the Classic Watchdog delivers significantly lower throughput than its of-watchdog counterpart, especially when the latter operates in HTTP mode. For instance, a Java function utilizing the Classic Watchdog might achieve approximately 91 requests per second (RPS), whereas the of-watchdog's "AfterBurn" mode can attain a remarkable 3000 RPS for the same function.⁷ Lastly, the Classic Watchdog's design involves buffering the entirety of the response from the function before transmitting it back to the client. This buffering mechanism inherently restricts its capacity to efficiently handle very large responses or streaming data that might exceed the container's available memory.¹¹

Understanding the Classic Watchdog's Design Implications

The Classic Watchdog's fundamental reliance on forking a new process for each request and its use of STDIO for communication⁶ are frequently likened to the historical Common Gateway Interface (CGI) model.⁷ CGI gained widespread adoption due to its simplicity and language agnosticism, allowing web servers to execute external programs without demanding that these programs possess HTTP awareness. However, the inherent inefficiency of creating a new process for every request quickly became a significant bottleneck for high-traffic web applications, ultimately leading to the development of more persistent models like FastCGI. The Classic Watchdog, in essence, embodies this historical CGI trade-off. It offers unparalleled simplicity and broad portability, enabling a diverse range of existing binaries and scripts to be seamlessly integrated into a serverless paradigm with minimal adaptation. However, this ease of adaptation comes at a direct cost to performance and resource efficiency, particularly for workloads demanding high concurrency or low latency. This makes the Classic Watchdog a pragmatic choice for "long-tail" functions—those that are invoked infrequently, are not performance-critical, or involve batch processing—or for integrating legacy CLI tools where the simplicity of adaptation outweighs the imperative for peak performance. The fact that it continues to be used in production environments⁸, despite its performance limitations, underscores the enduring value of its straightforward approach for specific use cases.

The `max_inflight` parameter, which directly limits the number of concurrent requests the

Classic Watchdog will accept ², plays a crucial role in its operational stability. Given that the Classic Watchdog forks a new process for *each request* ⁶, `max_inflight` effectively governs the maximum number of *concurrent function processes* that will be active within a single pod at any given moment. This mechanism establishes a fundamental, hard concurrency limit at the pod level. It prevents the watchdog from over-committing the pod's computational resources, such as CPU and memory, by attempting to spawn more processes than the underlying infrastructure or the function itself can reliably handle. This proactive limitation is vital for averting potential system crashes or severe performance degradation. Consequently, the incorrect configuration of `max_inflight` can have direct and immediate consequences on both performance and system stability. Setting this value too high for a resource-intensive function could lead to rapid resource exhaustion and instability. Conversely, setting it too low might result in the underutilization of the pod's allocated resources and an unnecessary proliferation of 429 Too Many Requests errors, even when the pod could technically manage a greater load. Therefore, `max_inflight` in the Classic Watchdog serves as a critical tuning parameter for ensuring the stability and predictable behavior of functions under varying loads. It functions as a rudimentary but highly effective form of admission control, providing essential backpressure and preventing a single overloaded function from adversely affecting the broader system.

3. The of-watchdog: Advanced Modes and Concurrency

Evolution and Core Design Principles: HTTP Server, Warm Processes, and Resource Reuse

The of-watchdog project, launched in October 2017, represents a significant evolution within the OpenFaaS ecosystem, serving as a more advanced and complementary alternative to the Classic Watchdog. Its primary innovation lies in its adoption of HTTP as the fundamental communication mechanism between the watchdog and the function, thereby transcending the limitations inherent in the Classic Watchdog's STDIO-based approach.³

The core design principles of of-watchdog were meticulously crafted to address the performance and resource efficiency challenges that are intrinsic to the Classic Watchdog's model. A central objective is to maintain function processes in an active, "warm" state between invocations. This capability dramatically reduces "cold start" latency, facilitates in-memory caching, and enables the establishment and persistence of connections, such as database connections.³ The ability for functions to "hold their states between calls" ¹⁹ is a direct and substantial benefit of this design. Furthermore, of-watchdog is explicitly engineered for use cases demanding the reuse of expensive resources. This includes scenarios where database connection pools, large machine learning models, or substantial data files need to be loaded only once and then efficiently shared across multiple requests, thereby eliminating redundant initialization overhead.³ The design also prioritizes the efficient streaming of large responses, overcoming the memory constraints associated with buffering

entire request or response bodies. This allows functions to process and transmit data that might otherwise exceed the container's allocated RAM or disk capacity.¹⁸ To cater to diverse interaction patterns with function code, of-watchdog introduces distinct operational modes, each optimized for specific use cases, providing clearer and more efficient abstractions.¹⁸ Architecturally, the of-watchdog functions as a sophisticated reverse proxy, implementing its own HTTP server that listens on port 8080. This architecture allows it to serve as the primary endpoint for a container within OpenFaaS, or even to operate as a standalone component.¹⁸

Detailed Concurrency Models by Mode

The of-watchdog offers several distinct operational modes, each characterized by a unique concurrency model and specific implications for function development and deployment.

HTTP Mode (`mode=http`)

This mode is the recommended and most efficient option, primarily intended for functions where the underlying programming language or framework provides its own HTTP server implementation. Examples include python3-http templates leveraging Flask, node* templates utilizing Express.js, or golang-middleware functions.¹⁰ Upon container startup, the of-watchdog forks the function's internal HTTP server process only once. All subsequent incoming requests directed to the watchdog are then forwarded as standard HTTP requests to this single, long-running, and "warm" function process within the container.¹¹

The benefits of HTTP mode are substantial. It is widely recognized as the "fastest option for high concurrency and throughput".¹⁸ Empirical benchmarks vividly demonstrate a dramatic increase in performance; for instance, a Java function can achieve approximately 3000 RPS in this mode, representing a monumental leap from the approximately 91 RPS observed with the Classic Watchdog.⁷ By eliminating the need to fork a new process for every request, HTTP mode facilitates "more efficient concurrency and RAM usage vs. forking model"¹⁸, as the overhead of process creation is effectively amortized across multiple invocations. It significantly reduces invocation latency, often achieving response times of "1ms or less" per request⁸, primarily by circumventing the performance penalty of cold starts. Crucially, this mode enables the persistence of expensive resources, such as database connections, large files, or machine learning models, for the entire lifecycle of the container. These resources can be initialized once and then seamlessly reused across all subsequent requests, leading to substantial performance gains and reduced operational costs.³

However, HTTP mode does introduce certain trade-offs. An additional HTTP hop is introduced in the request path (client -> watchdog -> function's internal HTTP server), which can contribute a minuscule amount of additional latency.¹⁸ Furthermore, the function developer assumes responsibility for implementing and correctly configuring an HTTP server within their function code. Frameworks like Express or Flask, while powerful, can exhibit unpredictable behavior if not managed appropriately in this context.¹⁸ Lastly, while more efficient per request, the persistent nature of the function process means it may occupy more memory between invocations compared to the Classic Watchdog's model, where processes are

terminated after each use.¹⁸

Serializing Fork Mode (mode=serializing)

This mode was specifically designed to emulate the behavior of the original Classic Watchdog, primarily to ensure backward compatibility with existing functions that anticipate the STDIO-based, fork-per-request model.¹⁸ It is particularly well-suited for retro-fitting traditional CGI application handlers.¹⁸ Similar to the Classic Watchdog, this mode forks a new process for each incoming request.¹⁸ The watchdog itself is multi-threaded, allowing it to manage multiple concurrent requests.¹⁸ However, a key aspect of its request processing is that it reads the *entire* HTTP request body into memory, potentially serializes or modifies it, and then writes this buffered data to the stdin pipe of the forked process. Conversely, the entire stdout output from the function is read into memory, potentially modified, and then written back as the HTTP response.¹⁸ Due to this inherent buffering mechanism, which requires reading both the entire request and response into memory, this mode is explicitly "limited to processing files sized as per available memory".¹⁸

Streaming Fork Mode (mode=streaming)

Designated as the default mode for of-watchdog¹⁸, the Streaming Fork mode was developed to overcome the memory limitations observed in the serializing mode. Its primary utility lies in enabling functions to efficiently handle very large request bodies and responses that might otherwise exceed the container's allocated memory, such as processing multiple gigabytes of video on a virtual machine with only 512MB of RAM.¹¹ Analogous to the serializing mode, it forks a new process for each request and operates as a multi-threaded component within the watchdog.¹¹ The critical distinction lies in its direct input/output hooking: input from the client is streamed directly to the function's stdin, and output generated from the function's stdout is transmitted back to the client as soon as it is printed, without full buffering.¹⁸ This direct streaming approach, while highly efficient for large data, imposes certain limitations. HTTP headers cannot be sent after the function has commenced execution due to the direct I/O pipeline. Furthermore, the HTTP response code is invariably 200 unless an issue arises with the process forking itself, meaning any errors that occur mid-flight within the function's execution must be detected and managed on the client side.¹⁸

max_inflight Implementation and Behavior Across of-watchdog Modes

Consistent with the Classic Watchdog, the of-watchdog also utilizes the max_inflight environment variable to enforce a hard limit on the number of concurrent requests that a single watchdog instance—and consequently, a single function pod—will actively process.¹³ When this configured limit is reached, the watchdog's /_ready health endpoint will respond with an HTTP 429 Too Many Requests status, providing a clear signal of backpressure to the upstream system.¹³ The OpenFaaS Pro queue-worker is specifically designed to automatically retry requests that encounter a 429 response, ensuring data integrity and system resilience.¹³

The interpretation and behavioral nuance of `max_inflight` subtly adapt depending on the active of-watchdog mode. In **HTTP mode**, `max_inflight` restricts the number of concurrent HTTP requests that the watchdog will forward to the *single, long-running function process*. The actual concurrency *within* that function process (e.g., how many requests a Node.js event loop or a Python Flask application can handle simultaneously) then falls under the purview and responsibility of the function's internal HTTP server implementation. Conversely, in both **Serializing Fork** and **Streaming Fork modes**, `max_inflight` directly limits the number of *concurrent process forks* that the watchdog manages, mirroring the behavior of the Classic Watchdog.

Key Enhancements and New Features

The of-watchdog introduces several significant enhancements and new features that bolster observability, control, and security within the OpenFaaS ecosystem. It improves structured logging and tracing capabilities by allowing the correlation of requests. Specifically, it can append the X-Call-Id header, a correlation ID propagated from the OpenFaaS Gateway, to its own HTTP log messages when properly configured. Developers also retain the flexibility to output their own log lines in JSON format by simply disabling the default log prefix.¹⁸ A notable improvement is the ability to override the default `exec_timeout` for individual requests. This is achieved by passing an X-Timeout HTTP header with a Go duration, providing more granular control over function execution time on a per-invocation basis, provided the specified timeout is equal to or shorter than the configured `exec_timeout`.¹⁸ Beyond basic health checks, of-watchdog offers dedicated `/_health` and `/_ready` endpoints. The `/_ready` endpoint is particularly valuable for Kubernetes readiness probes, as it can return a 429 status if `max_inflight` is configured and the maximum number of connections is met, enabling the orchestrator to apply backpressure or intelligently re-route traffic.¹³ For OpenFaaS for Enterprises customers, of-watchdog integrates built-in support for validating JSON Web Tokens (JWT) passed as Bearer tokens in the Authorization header, significantly enhancing the security posture of function invocations.⁶

Understanding of-watchdog's Performance Acceleration

The stated objectives of of-watchdog explicitly center on "keeping function processes warm," achieving "lower latency," enabling "caching," facilitating "persistent connections," and supporting "streaming of large responses".¹⁸ Quantitative performance benchmarks provide compelling evidence of its success in these areas. Latency, which could be over 100ms with the Classic Watchdog, is dramatically reduced to "1ms or less" with of-watchdog.⁸ Throughput is also substantially increased; for instance, a Java function can achieve approximately 3000 RPS, a significant leap from the ~91 RPS observed with the Classic Watchdog.⁷ Furthermore, of-watchdog promises "more efficient concurrency and RAM usage".¹⁸ This body of evidence strongly indicates that of-watchdog is not merely a simple execution layer but a critical performance optimization component. Its design actively addresses the most significant performance bottlenecks in serverless computing, such as cold starts and the inefficient reuse of resources. By allowing functions to remain "warm" and by facilitating the persistence

of expensive resources like database connections or machine learning models across invocations, of-watchdog fundamentally transforms the performance profile of OpenFaaS functions. This architectural shift enables OpenFaaS to support high-throughput, low-latency workloads that would be impractical or inefficient with the Classic Watchdog.

4. Comparative Analysis: Key Distinctions and Performance Implications

The architectural divergence between the Classic Watchdog and of-watchdog manifests across several critical dimensions, each bearing significant implications for function performance, resource utilization, and suitability for various workloads.

Communication Protocol (STDIO vs. HTTP)

The most fundamental difference lies in their communication protocols. The Classic Watchdog relies on standard input/output (STDIO) for interaction with the function process.⁴ It marshals HTTP requests into stdin and reads responses from stdout. This design is inherently simple and highly portable, allowing any binary or script that can read from stdin and write to stdout to become a function.⁸ This approach is akin to the traditional CGI model.⁷

In contrast, of-watchdog primarily utilizes HTTP for communication between itself and the function code, particularly in its recommended HTTP mode.³ This necessitates that the function itself implements an HTTP server (e.g., using Flask for Python or Express.js for Node.js).¹⁰ While introducing an additional HTTP hop, this paradigm shift enables more advanced features and performance optimizations. The of-watchdog also retains STDIO-like behavior in its serializing and streaming fork modes for backward compatibility and specific use cases.¹⁸

Process Management and Warm vs. Cold Starts

The distinction in process management is central to their performance characteristics. The Classic Watchdog adheres strictly to a "process-per-request" model, where a new function process is forked for every incoming invocation.³ While providing excellent process isolation¹¹, this leads to significant "cold start" overhead, especially for languages with slower initialization times like Java or Node.js.⁷ This overhead can represent a substantial portion of the total invocation latency.¹⁷

of-watchdog, particularly in its HTTP mode, fundamentally alters this by keeping the function process "warm" between invocations.³ It forks the function's HTTP server process only once upon container startup, and subsequent requests are forwarded to this long-running process.¹¹ This significantly reduces latency, often to "1ms or less"⁸, and enables the persistence and reuse of expensive resources like database connections or machine learning models across multiple requests.³ Even its fork modes (serializing and streaming) offer multi-threading within the watchdog itself, improving concurrency management compared to the Classic's simpler model.¹⁸

Concurrency Model and Resource Utilization

The concurrency model of the Classic Watchdog, while supporting concurrent *HTTP connections* via its Golang server, executes functions in a "fork-per-request" manner. This means each concurrent request results in a new, isolated function process.² This model can lead to higher CPU and memory consumption due to the constant creation and teardown of processes.¹⁸ The `max_inflight` parameter directly limits the number of these concurrent processes within a pod, acting as a hard concurrency gate and backpressure mechanism.² of-watchdog offers more sophisticated concurrency. In HTTP mode, `max_inflight` limits concurrent requests forwarded to a single, warm function process, allowing the function's internal HTTP server to manage its own concurrency efficiently.¹³ This yields "faster option for high concurrency and throughput" and "more efficient concurrency and RAM usage".¹⁸ In its fork modes, while still forking per request, of-watchdog is multi-threaded and can handle large request bodies via streaming, preventing memory exhaustion for large payloads.¹⁸ The `max_inflight` parameter continues to provide crucial backpressure across all of-watchdog modes, returning 429 Too Many Requests when limits are hit.¹³

Ideal Workloads and Use Cases

The Classic Watchdog is ideally suited for functions that do not inherently expose an HTTP server, such as Bash scripts, PowerShell scripts, or existing command-line tools.⁸ It is also a good choice for simple, stateless scripts, initial development, or batch processing where the overhead of process creation is acceptable.⁶ Its primary advantage is its simplicity and broad compatibility.

of-watchdog is designed for performance-critical workloads and functions that benefit from persistent state or resource reuse. Its HTTP mode is optimal for languages and frameworks with built-in HTTP servers (e.g., Python Flask, Node.js Express), enabling high throughput, low latency, and efficient use of resources like database connections or ML models.³ The streaming fork mode is specifically tailored for processing large data streams that exceed container memory limits, such as video processing.¹⁸ of-watchdog is increasingly popular and used in modern OpenFaaS templates.³

Performance Benchmarks (Latency, Throughput, Memory/CPU)

Performance benchmarks consistently highlight the superior performance of of-watchdog, particularly in its HTTP mode, over the Classic Watchdog. The Classic Watchdog, due to its fork-per-request model, exhibits noticeable cold start latencies and lower throughput. For instance, a Java function might achieve only around 91 RPS with the Classic Watchdog.⁷ This is a direct consequence of the overhead involved in spawning a new process for each invocation, which can significantly impact overall latency, with cold starts alone potentially accounting for up to 80% of total latency.¹⁷

In contrast, of-watchdog in HTTP mode drastically reduces latency, often to "1ms or less" per request, and delivers significantly higher throughput, with a Java function potentially reaching

approximately 3000 RPS.⁷ This substantial performance improvement stems from its ability to keep function processes warm and reuse resources across requests, thereby amortizing initialization costs and reducing the per-invocation overhead.³ While specific memory and CPU consumption figures vary based on function logic and workload, the of-watchdog's HTTP mode generally offers "more efficient concurrency and RAM usage vs. forking model"¹⁸, as it avoids the continuous process creation overhead. Both watchdogs provide Prometheus metrics, including inflight requests, request count, and latency, which are crucial for autoscaling and monitoring resource usage.²⁰

5. Conclusions and Recommendations

The OpenFaaS watchdog components, the Classic Watchdog and of-watchdog, represent distinct evolutionary stages in optimizing serverless function execution within containerized environments. The Classic Watchdog, with its STDIO-based, process-per-request model, prioritizes simplicity and broad compatibility, effectively transforming any CLI or script into an HTTP endpoint. This design, while robust for simple, stateless, or batch workloads, inherently suffers from cold start latencies and higher resource overhead due to the constant forking of new processes. Its `max_inflight` parameter serves as a crucial, albeit rudimentary, concurrency gate, preventing resource exhaustion by limiting the number of simultaneous function processes. This mechanism is vital for maintaining stability and predictable behavior under load.

The of-watchdog emerged as a performance-oriented evolution, fundamentally shifting to an HTTP-based communication model. Its primary innovation lies in keeping function processes "warm," thereby dramatically reducing cold start latencies and enabling the persistence and reuse of expensive resources across invocations. This component is not merely an execution shim; it is a critical performance accelerator, evidenced by significant improvements in latency (down to sub-millisecond) and throughput (orders of magnitude higher). Its multi-mode design—encompassing HTTP, Serializing Fork, and Streaming Fork modes—offers strategic flexibility, allowing developers to select the optimal execution model based on specific function characteristics, such as the need for high concurrency, large data streaming, or backward compatibility. The `max_inflight` parameter continues to provide essential backpressure across all of-watchdog modes, ensuring resource governance.

Recommendations for Choosing Between Watchdogs:

- **For maximum simplicity and compatibility with non-HTTP-aware binaries/scripts:** The **Classic Watchdog** remains a viable choice. This is ideal for wrapping existing command-line tools, simple Bash scripts, or for functions where cold start latency is not a critical performance factor (e.g., infrequent invocations, background tasks).
- **For high-performance, low-latency, and resource-efficient functions:** The **of-watchdog in HTTP mode** is the unequivocally recommended option. This mode is best suited for modern functions written in languages with robust HTTP server frameworks (e.g., Python with Flask, Node.js with Express, Go) and for workloads requiring persistent connections, in-memory caching, or the reuse of expensive resources (e.g., database connections, machine learning models).

- **For handling very large request or response bodies that exceed container memory:** The **of-watchdog in Streaming Fork mode** is the appropriate choice. This mode enables efficient processing of large data streams by avoiding full buffering.
- **For backward compatibility with existing Classic Watchdog functions that require buffering:** The **of-watchdog in Serializing Fork mode** can be utilized, though its memory limitations for large payloads should be noted.

In summary, the choice between the Classic Watchdog and of-watchdog hinges on a careful assessment of function requirements, performance expectations, and resource utilization goals. While the Classic Watchdog offers simplicity and broad adaptability, the of-watchdog provides a sophisticated, performance-optimized foundation for building modern, highly scalable, and efficient serverless applications on OpenFaaS.

Works cited

1. OpenFaaS: Introduction, accessed June 4, 2025, <https://docs.openfaas.com/>
2. accessed January 1, 1970, <https://github.com/openfaas/classic-watchdog/blob/master/main.go>
3. Watchdog - OpenFaaS, accessed June 4, 2025, <https://docs.openfaas.com/architecture/watchdog/>
4. Watchdog - OpenFaaS, accessed June 4, 2025, <https://ericstoekl.github.io/faas/developer/watchdog/>
5. Serverless Open-Source Frameworks: OpenFaaS, Knative, & more | CNCF, accessed June 4, 2025, <https://www.cncf.io/blog/2020/04/13/serverless-open-source-frameworks-openfaas-knative-more/>
6. Classic Watchdog for OpenFaaS - GitHub, accessed June 4, 2025, <https://github.com/openfaas/classic-watchdog>
7. OpenFaaS accelerates serverless Java with AfterBurn - Alex Ellis' Blog, accessed June 4, 2025, <https://blog.alexellis.io/openfaas-serverless-acceleration/>
8. Dude where's my coldstart? | OpenFaaS - Serverless Functions Made Simple, accessed June 4, 2025, <https://www.openfaas.com/blog/what-serverless-coldstart/>
9. Turn Any CLI into a Function with OpenFaaS - Alex Ellis' Blog, accessed June 4, 2025, <https://blog.alexellis.io/cli-functions-with-openfaas/>
10. Custom - OpenFaaS, accessed June 4, 2025, <https://docs.openfaas.com/languages/custom/>
11. OpenFaaS - Run Containerized Functions On Your Own Terms, accessed June 4, 2025, <https://iximiuz.com/en/posts/openfaas-case-study/>
12. OpenFaaS Classic templates - GitHub, accessed June 4, 2025, <https://github.com/openfaas/templates>
13. Auto-scaling your functions - OpenFaaS, accessed June 4, 2025, <https://docs.openfaas.com/architecture/autoscaling/>
14. How to process your data the resilient way with back pressure | OpenFaaS, accessed June 4, 2025, <https://www.openfaas.com/blog/limits-and-backpressure/>

15. Function invocations - OpenFaaS, accessed June 4, 2025,
<https://docs.openfaas.com/architecture/invocations/>
16. Create functions - OpenFaaS, accessed June 4, 2025,
<https://docs.openfaas.com/cli/templates/>
17. Mitigating the cold start issue in Serverless using Reinforcement learning -
Utrecht University Student Theses Repository Home, accessed June 4, 2025,
[https://studenttheses.uu.nl/bitstream/handle/20.500.12932/48676/Rosen_Kosakov
Master_Thesis_Final_Version.pdf?sequence=1&isAllowed=y](https://studenttheses.uu.nl/bitstream/handle/20.500.12932/48676/Rosen_Kosakov_Master_Thesis_Final_Version.pdf?sequence=1&isAllowed=y)
18. openfaas/of-watchdog: Reverse proxy for STDIO and HTTP microservices -
GitHub, accessed June 4, 2025, <https://github.com/openfaas/of-watchdog>
19. Use Hazelcast with OpenFaaS, accessed June 4, 2025,
<https://docs.hazelcast.com/tutorials/openfaas-hz-client>
20. How to scale OpenFaaS Functions with Custom Metrics, accessed June 4, 2025,
<https://www.openfaas.com/blog/custom-metrics-scaling/>
21. Metrics - OpenFaaS, accessed June 4, 2025,
<https://docs.openfaas.com/architecture/metrics/>